Amazing Hands

Bob the Magician has a deck of $2N$ cards, numbered from 1 to $2N$. Initially, the cards are arranged in the order $1, 2, \ldots, 2N$ from top to bottom.

Bob can perform two types of operations on the deck: **Shuffle** and **Cut**. Let $A_1, A_2, \ldots, A_{2N}$ denote the cards in the deck from top to bottom before an operation:

1. **Shuffle**: The deck is split into two halves of N cards ($A_1, \ldots, A_N$ and $A_{N+1}, \ldots, A_{2N}$) and perfectly interleaved, keeping the top and bottom cards in place. The new order from top to bottom becomes
$$A_1, A_{N+1}, A_2, A_{N+2}, \ldots, A_N, A_{2N}.$$
2. **Cut**: Choose an integer k ($0 \leq k \leq 2N-1$), take the top k cards, and move them to the bottom of the deck without changing their relative order. (If $k = 0$, the deck remains unchanged.) The new order from top to bottom becomes
$$A_{k+1}, A_{k+2}, \ldots, A_{2N}, A_1, \ldots, A_k.$$

Bob performs the Shuffle operation several (possibly zero) times, followed by exactly one Cut operation with some chosen integer k. He then reveals that the top two cards of the deck are X and Y (with X being the top card and Y being the second card).

Your task is to determine the number on the **third card** in the deck, or report that there are multiple possible answers. It is guaranteed that there exists at least one sequence of operations resulting in X and Y as the top two cards.

Input

The input consists of a single line containing three integers N, X, and Y ($2 \leq N \leq 10^9$, $1 \leq X, Y \leq 2N$, and $X \neq Y$) – half the size of the deck, and the numbers on the top two cards after Bob's operations, respectively.

Output

Output a single integer – the number on the third card in the deck. If there are multiple possible answers, output -1.

Examples

input	output
4 5 7	2
3 6 1	-1

Explanation

In the first sample, the deck has $2N = 8$ cards, initially in the order $1, 2, 3, 4, 5, 6, 7, 8$.

- After 1 shuffle, the order is $1, 5, 2, 6, 3, 7, 4, 8$.
- After 2 shuffles, the order is $1, 3, 5, 7, 2, 4, 6, 8$.

If Bob performs 2 shuffles followed by a cut with $k = 2$ (moving cards 1 and 3 to the bottom), the deck becomes $5, 7, 2, 4, 6, 8, 1, 3$. The top two cards are $X = 5$ and $Y = 7$, and the third card is 2. In any valid sequence of operations resulting in 5 and 7 as the top two cards, the third card is uniquely determined to be 2.

In the second sample:

- With no shuffles, a cut with $k = 5$ produces $6, 1, 2, 3, 4, 5$.
- After one shuffle, the same cut produces $6, 1, 4, 2, 5, 3$.

Since multiple third cards are possible, the answer is -1.

Boundless Power

As energy prices fluctuate frequently in the modern world, Vladimir is considering an upgrade to his home power system. Currently, he has solar panels installed on his roof, and he can also purchase electricity from the grid to cover his needs.

Based on historical data, he has predicted the following information for each of the next N hours (numbered from 1 to N):

- P_i: the price of electricity from the grid in hour i (in USD per MWh);
- R_i: the amount of energy he requires in hour i (in MWh);
- S_i: the amount of energy generated by his solar panels in hour i (in MWh).

Vladimir is considering buying a battery with a capacity of B MWh to store energy for use over these N hours. Initially, the battery is empty (it stores 0 MWh).

In each hour i, Vladimir has access to the energy currently stored in the battery, the S_i MWh generated by his solar panels, and any non-negative integer amount of electricity he chooses to purchase from the grid at the rate of P_i USD per MWh.

During hour i, Vladimir must perform the following actions:

1. Satisfy his demand: consume exactly R_i MWh of available energy.
2. Optionally sell energy back to the grid: the power company offers M buyback plans for hour i. Vladimir may select **at most one** plan j ($1 \le j \le M$), providing exactly $D_{i,j}$ MWh of energy to the grid in exchange for $C_{i,j}$ USD.
3. Store remaining energy: store any amount of leftover energy (from the grid, solar panels, and leftover energy from the battery) in the battery, up to its capacity B MWh, to be available in subsequent hours. Any remaining excess energy is discarded.

All quantities of energy bought, consumed, sold, and stored must be non-negative integers.

Help Vladimir determine the minimum net cost (total money spent purchasing electricity minus total money earned from buyback plans) required to satisfy his energy needs over all N hours. If he earns more money than he spends, the net cost can be negative.

Input

The first line contains three integers N, M, and B ($1 \le N \le 1\,000$, $1 \le M \le 10$, $1 \le B \le 20$) – the number of hours, the number of buyback plans per hour, and the capacity of the battery in MWh.

Each of the next N lines contains three integers P_i, R_i, and S_i ($1 \le P_i, R_i, S_i \le 100$) – the grid electricity price, the energy demand, and the solar generation in hour i.

Each of the next N lines contains M integers $C_{i,1}, C_{i,2}, \ldots, C_{i,M}$ ($1 \le C_{i,j} \le 10\,000$) – the revenue in USD from each buyback plan in hour i.

Each of the next N lines contains M integers $D_{i,1}, D_{i,2}, \ldots, D_{i,M}$ ($1 \le D_{i,j} \le 100$) – the amount of energy in MWh required by each buyback plan in hour i.

Output

Output a single integer – the minimum net cost to satisfy Vladimir's energy needs for the next N hours.

Examples

input	output
2 1 10 4 3 3 1 5 3 4 9 2 3	-4
3 4 5 1 5 3 9 4 3 10 6 3 1 3 4 9 1 3 6 9 1 3 6 30 1 2 3 4 1 2 3 4 1 2 3 4	1
3 2 2 1 3 6 4 6 3 10 6 3 4 10 2 9 10 20 1 10 2 4 1 3	18

Crazy Cards

Claire is a big fan of the card game Crazy Cards. A deck of Crazy Cards consists of N cards. Unlike standard playing cards, each card has a figure on both sides.

Initially, all N cards lie on the table with their front side facing up. Claire knows the figures on both sides of every card, and she wants to maximize the total number of distinct figures visible on the table. To achieve this, she can choose any subset of cards to flip, so that their back side is facing up instead.

Help Claire determine the maximum number of distinct figures that can be visible on the table after choosing which cards to flip.

Input

The first line contains a single integer N ($1 \leq N \leq 200\,000$) – the number of cards in the deck.

The second and third lines describe the figures on each card, represented by integers between 0 and 10^6:

- The second line contains N integers $F_1, F_2, \ldots, F_N$ ($0 \leq F_i \leq 10^6$) – the figures on the front side of the cards.
- The third line contains N integers $B_1, B_2, \ldots, B_N$ ($0 \leq B_i \leq 10^6$) – the figures on the back side of the cards.

Output

Output a single integer – the maximum number of distinct figures that can be visible on the table.

Examples

input	output
5 0 1 0 0 1 1 0 2 3 1	4
2 3 5 5 1	2
3 0 1 0 1 0 2	3

Explanation

In the first sample, flipping the third and fourth cards leaves the visible figures as $0, 1, 2, 3, 1$. There are 4 distinct figures (0, 1, 2, and 3), which is the maximum possible.

Dungeon Deluxe

You are playing the brand-new sequel to your favorite computer game, *Infernal Creatures' Paradise: Couchcaster*. In the game, you are developing a hero that must complete a grueling challenge consisting of N encounters in order, numbered from 1 to N. Your hero has two statistics: *charisma* and *endurance*.

In each encounter, the hero faces a trial with a demonic creature. To pass the trial in encounter i, the hero must have at least as much charisma and endurance as the creature (C_i and E_i, respectively). If the hero passes the trial, he can increase one of his statistics by 1 point and proceed to the next encounter. Otherwise, the demonic creature will completely suck the life essence out of the hero, and the game ends.

The developers are greedy and designed the game to be impossible to complete without spending money on in-game purchases: the hero starts with 0 charisma and 0 endurance. Before the game starts, the player can purchase points to set the hero's initial statistics for a price of 1 coin per point.

You are given the statistics of the demonic creatures in each encounter. Your task is to determine the minimum number of coins you need to spend on the hero's initial statistics so that he can complete all N encounters.

Input

The first line of the input contains a single integer N ($1 \leq N \leq 50$) – the number of encounters.

Each of the next N lines contains two integers C_i and E_i ($1 \leq C_i, E_i \leq 10^9$) – the charisma and endurance of the demonic creature in the i-th encounter.

Output

Output a single integer – the minimum number of coins you need to spend on the hero's initial statistics.

Examples

input	output
3 4 4 5 1 1 5	8
3 4 4 1 6 6 1	10
2 33 33 34 34	67

Explanation

In the first sample, the player can spend 8 coins to start with 4 charisma and 4 endurance:

- Encounter 1: The creature requires $(4, 4)$. The hero passes the trial and increases his charisma by 1, bringing his stats to $(5, 4)$.
- Encounter 2: The creature requires $(5, 1)$. The hero passes the trial and increases his endurance by 1, bringing his stats to $(5, 5)$.
- Encounter 3: The creature requires $(1, 5)$. The hero passes the trial and completes the challenge.

It is impossible to complete all encounters by spending fewer than 8 coins.

Enchanted Terminal

While exploring a forgotten library, an archaeologist discovers an ancient magical terminal. Initially, the terminal screen is completely empty, and its keyboard features only two buttons, labeled `0` and `1`.

Through experimentation, the archaeologist realizes that the terminal is enchanted:

- Pressing the `0` button appends the character `0` to the right end of the string currently displayed on the screen.
- Pressing the `1` button appends the character `1` to the right end of the string, and then immediately **reverses** the entire string on the screen.

For example, starting from an empty screen, pressing `0`, then `1`, then `1`, and finally `0` produces the following sequence of strings on the screen:

$$\text{empty} \xrightarrow{0} \texttt{0} \xrightarrow{1} \texttt{10} \xrightarrow{1} \texttt{101} \xrightarrow{0} \texttt{1010}.$$

Because each button press adds exactly one character to the screen, producing a string of length N requires exactly N button presses. Given a target binary string S of length N, determine whether it is possible to display S on the screen starting from an empty screen. If it is possible, output any sequence of N button presses that produces S.

Input

The first line contains a single integer N ($1 \leq N \leq 100\,000$) – the length of the target binary string.

The second line contains a binary string S of length N consisting only of the characters `0` and `1`.

Output

If it is impossible to produce the string S, output a single line containing the word `IMPOSSIBLE`.

Otherwise, output a single line containing a binary string of length N – a sequence of button presses in chronological order from first to last that produces S on the screen. If there are multiple valid sequences, output any of them.

Examples

input	output
4 1010	0110
2 01	IMPOSSIBLE
5 11001	01011

Explanation

In the first sample, the target string is `1010`. Two sequences of button presses can produce `1010`: `0110` and `1010`. Either of these outputs would be accepted.

In the second sample, the target string is `01`. After the first button press, the screen shows either `0` or `1`. From `0`, pressing `0` yields `00` and pressing `1` yields `10`. From `1`, pressing `0` yields `10` and pressing `1` yields `11`. Thus, `01` can never be formed.

Folded Topography

To prepare for an upcoming expedition, a cartographer needs to fold a large rectangular map so that it fits neatly into a travel pouch. Initially, the map is a grid of N rows and M columns of unit cells, where every cell starts with a thickness of 1.

The cartographer performs a sequence of Q folding operations, one after another. Each operation specifies one of the four sides of the map – `T` (top), `B` (bottom), `L` (left), or `R` (right) – and a positive integer distance d, which is strictly less than the current dimension of the map along the folding direction.

When a fold of distance d is performed from a given side:

- A crease line is made at distance d from that side, and the outermost d rows (for `T` or `B`) or d columns (for `L` or `R`) are folded inward over the crease line onto the remaining part of the map.
- For each $1 \leq k \leq d$, the k-th row or column from the crease on the folded flap lands directly on top of the k-th row or column on the other side of the crease, and their cell thicknesses are added together.
- If d is strictly greater than the remaining length on the other side of the crease line, the folded flap extends past the opposite edge of the map.
- The crease line becomes the new outer boundary on that side. Along the folded direction, the new dimension of the map becomes $\max(d, \text{current dimension} - d)$.

For example, suppose a map currently has 1 row and 5 columns with cell thicknesses $[1, 1, 1, 1, 1]$ from left to right:

- Folding from the left with distance $d = 2$ (`L 2`) places the crease between columns 2 and 3. Column 2 lands on column 3, and column 1 lands on column 4. The new row has $\max(2, 5-2) = 3$ columns with thicknesses $[2, 2, 1]$.
- If we instead fold $[1, 1, 1, 1, 1]$ from the left with distance $d = 4$ (`L 4`), the crease is between columns 4 and 5. Column 4 lands on column 5, while columns $3, 2, 1$ extend beyond the right edge. Ordered from left to right (starting from the crease), the new row has $\max(4, 5-4) = 4$ columns with thicknesses $[2, 1, 1, 1]$.

After all Q folding instructions have been executed, determine the final number of rows and columns of the map, as well as the maximum thickness of any single cell.

Input

The first line contains three integers N, M, and Q ($1 \leq N, M \leq 2000$, $1 \leq Q \leq 2000$) – the initial number of rows, the initial number of columns, and the number of folding instructions.

Each of the next Q lines contains a character $c_i \in \{\texttt{T}, \texttt{B}, \texttt{L}, \texttt{R}\}$ and an integer d_i, describing the side and distance of the i-th fold. It is guaranteed that at the moment the i-th fold is performed:

- if $c_i \in \{\texttt{T}, \texttt{B}\}$, then $1 \leq d_i < H$, where H is the current number of rows of the map;

- if $c_i \in \{\texttt{L}, \texttt{R}\}$, then $1 \leq d_i < W$, where W is the current number of columns of the map.

Output

Output three space-separated integers on a single line: the final number of rows, the final number of columns, and the maximum thickness of any single cell after all Q folds.

Examples

input	output
4 5 3 L 2 T 1 R 2	3 2 8
1 5 1 L 4	1 4 2
4 4 4 T 2 B 1 L 2 R 1	1 1 16

Explanation

In the first sample, the map starts as a 4×5 grid of unit thickness. Below is the state of the grid after each folding operation:

$$\begin{pmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{pmatrix} \xrightarrow{\text{L 2}} \begin{pmatrix} 2 & 2 & 1 \\ 2 & 2 & 1 \\ 2 & 2 & 1 \\ 2 & 2 & 1 \end{pmatrix} \xrightarrow{\text{T 1}} \begin{pmatrix} 4 & 4 & 2 \\ 2 & 2 & 1 \\ 2 & 2 & 1 \end{pmatrix} \xrightarrow{\text{R 2}} \begin{pmatrix} 2 & 8 \\ 1 & 4 \\ 1 & 4 \end{pmatrix}$$

After all 3 folding instructions are executed, the final grid has dimensions 3×2, and the maximum thickness of any single cell is 8 (at the top-right cell).

Geeky Cheese!

Another year, another ICPC World Finals! As university delegations arrive at the venue, the traditional group photo session is taking place. Your delegation consists of N contestants, where N is an **even** integer.

The contestants must stand in a straight line for the photo. To make the picture memorable, half of the contestants are wearing a T-shirt with '`(`' printed on it, and the other half are wearing a T-shirt with '`)`' printed on it. You decide to arrange the contestants so that the sequence of their T-shirts from left to right forms a **valid parenthesis sequence**. A parenthesis sequence is valid if it is empty, or if it can be written as AB where A and B are valid parenthesis sequences, or as (A) where A is a valid parenthesis sequence.

Before everyone shouts "Cheese!", you realize that you cannot decide on just one arrangement. Instead, you ask the photographer to take multiple photos so that **every** valid parenthesis sequence of length N is captured in exactly one photo. The photographer is in a hurry, so you must transition between consecutive arrangements as quickly as possible: between any two consecutive photos, you may only swap the positions of two contestants.

Your task is to find an initial arrangement of contestants and a sequence of swaps such that every valid parenthesis sequence of length N appears in exactly one photo, or determine that it is impossible.

Input

The first line of the input contains a single integer N ($2 \leq N \leq 20$, with N even) – the number of contestants in your delegation.

Output

If it is impossible to visit every valid parenthesis sequence via single swaps, output a single line containing `IMPOSSIBLE`.

Otherwise:

- On the first line, output a string of length N consisting of `(` and `)` representing the initial valid parenthesis sequence.
- On each subsequent line, output two integers a and b ($1 \leq a, b \leq N$, $a \neq b$) separated by a space, indicating that the contestants at 1-based positions a and b swap places to form the next photo's valid parenthesis sequence.

If there are multiple valid solutions, output any of them.

Examples

input	output
6	(()()) 3 4 2 3 4 5 2 3

Explanation

For $N = 6$, there are 5 valid parenthesis sequences in total. The sample output produces all of them as follows:

1. Initial sequence: `(()())`
2. Swap positions 3 and 4: `(()())`
3. Swap positions 2 and 3: `()(())`
4. Swap positions 4 and 5: `()()()`
5. Swap positions 2 and 3: `(())()`

Each of the 5 valid sequences is shown in exactly one photo.

Hornet Nests

This is an interactive problem. Your program communicates with the grader by sending and receiving messages through the standard input and output streams.

While enjoying the warm summer sun in the garden, Cleopatra Millan noticed an unusually high number of hornets flying around. After careful investigation, she realized that there are exactly N hornets in her garden ($3 \leq N \leq 1000$), numbered from 0 to $N-1$.

The hornets are distributed across several nests, where every hornet belongs to exactly one nest:

- Each hornet i has a favorite hornet H_i ($0 \leq H_i < N$). The favorite hornets are distinct; that is, $H = [H_0, H_1, \ldots, H_{N-1}]$ is a permutation of $0, 1, \ldots, N-1$.
- Hornet i lives in the same nest as hornet H_i. (If $H_i = i$, hornet i may live in a nest by itself.)
- The hornets live in as many nests as possible subject to this condition. In other words, each nest corresponds to a disjoint cycle in the permutation H.

Cleopatra does not know the favorite hornets initially.

Cleopatra is a hornet whisperer, so she can persuade hornets to swap their favorites. In one operation, she can choose two distinct hornets x and y ($0 \leq x, y < N$, $x \neq y$) and persuade them to swap favorites: hornet x's favorite becomes the old H_y, and hornet y's favorite becomes the old H_x. After the swap, the nest structure updates according to the new permutation, and Cleopatra walks around the garden to count the total number of nests.

Cleopatra knows that nests containing multiple hornets are dangerous. She wants to reconfigure the hornets' favorites so that every hornet lives by itself (meaning $H_i = i$ for all i, resulting in exactly N nests).

Your task is to help Cleopatra achieve this goal using at most 30 000 operations.

Interaction

Your program should first read a single integer N ($3 \leq N \leq 1000$) from the standard input — the number of hornets. Note that the initial number of nests is **not** provided directly.

Then, your program may perform a sequence of operations. To perform an operation, print a single line to the standard output in the following format:

`change` x y

where x and y are distinct integers with $0 \leq x, y < N$.

In response to each operation, the grader will provide a single integer on the standard input — the total number of nests after the swap.

Your goal is to reach a state where the number of nests is N. Once this is achieved, your program should output:

`done`

and terminate immediately.

If your program violates any interaction rules (such as an invalid command format, coordinates out of bounds, $x = y$, or exceeding 30 000 operations), your submission will receive a `Wrong Answer` verdict.

After printing each command (including `done`), **do not forget to print a newline character** and **flush the standard output**. Otherwise, your solution may time out. To flush the output, use:

- `fflush(stdout)` in C or `cout << flush` in C++;
- `Console.Out.Flush()` in C#;
- `System.out.flush()` in Java;
- `sys.stdout.flush()` in Python;

Example

Consider a case where $N = 3$ and the initial favorite hornets are $H = [2, 0, 1]$ (all 3 hornets belong to a single cycle $0 \to 2 \to 1 \to 0$, forming 1 nest). An example of a valid interaction is shown below:

input	output
3 2 1 2 3	 change 0 1 change 0 1 change 0 1 change 1 2 done

Explanation of the interaction:

1. The program reads $N = 3$.
2. The program sends `change 0 1`. The permutation becomes $H = [0, 2, 1]$. Hornet 0 forms its own nest ($0 \to 0$), and hornets 1 and 2 form another ($1 \to 2 \to 1$). The grader responds with 2.
3. The program sends `change 0 1` again. The permutation reverts to $H = [2, 0, 1]$. The grader responds with 1.
4. The program sends `change 0 1` once more. The permutation returns to $H = [0, 2, 1]$. The grader responds with 2.
5. The program sends `change 1 2`. The permutation becomes $H = [0, 1, 2]$ (three singleton cycles). The grader responds with 3.

Since there are now $N = 3$ nests, every hornet lives by itself, and the program prints `done` and terminates.

In the Heat

In the Great Hungarian Plain, there is a village with N houses. The village can be represented as a circular loop of length L, with positions along the loop numbered from 0 to $L-1$ in the clockwise direction (so position 0 immediately follows position $L-1$). The houses are located at integer coordinates $P_0, P_1, \ldots, P_{N-1}$ given in strictly increasing order, i.e., $0 \leq P_0 < P_1 < \ldots < P_{N-1} < L$.

The owner of house i ($0 \leq i < N$) owns horse i. Due to the intense heat and the lack of water that summer, all horses simultaneously went mad and began running along the loop. Each horse i starts at its house at position P_i and runs at a constant speed of 1 unit per second, either clockwise (towards increasing positions) or counterclockwise (towards decreasing positions). When two horses collide, they immediately reverse their directions without any loss of time and continue running at the same speed. Horses cannot pass through each other.

Amid the chaos, the villagers are trying to locate their horses. They have Q queries, each asking for the position of a specific horse C_j after T_j seconds from the start. The villagers contact the famous computer scientist Professor Tibor von Minerale to help them. The professor solved the problem in no time, and now you are asked to do the same.

Input

The first line contains two integers N and L ($2 \leq N \leq 1\,000\,000$, $N \leq L \leq 10^9$) – the number of houses and the length of the loop, respectively.

The second line contains N integers $P_0, P_1, \ldots, P_{N-1}$ ($0 \leq P_i < L$) – the positions of the houses along the loop in increasing order.

The third line contains N integers $D_0, D_1, \ldots, D_{N-1}$ ($D_i \in \{-1, 1\}$) – the initial directions of the horses, where 1 means clockwise and -1 means counterclockwise.

The fourth line contains a single integer Q ($1 \leq Q \leq 100\,000$) – the number of queries.

Each of the next Q lines contains two integers C_j and T_j ($0 \leq C_j < N$, $0 \leq T_j \leq 10^9$) – the index of the horse and the elapsed time in seconds.

Output

Output Q lines, where the j-th line contains a single integer – the position of horse C_j after T_j seconds.

Examples

input	output
3 6 0 2 5 -1 1 1 3 0 2 2 1 1 5	1 5 4
3 12461 5049 6138 6513 1 -1 -1 3 1 178432 0 1414122 2 24922	9027 484 5049

Explanation

In the first sample, the loop has length $L = 6$. The horses start at positions $P_0 = 0$, $P_1 = 2$, and $P_2 = 5$:

- Query `0 2` (Horse 0 after 2 seconds): Horse 0 starts at position 0 running counterclockwise, and horse 2 starts at position 5 running clockwise. At $t = 0.5$, they collide at position 5.5 and reverse directions: horse 0 now moves clockwise and horse 2 moves counterclockwise. Moving clockwise, horse 0 reaches position 0 at $t = 1.0$ and position 1 at $t = 2.0$. The answer is 1.
- Query `2 1` (Horse 2 after 1 second): After colliding at $t = 0.5$ at position 5.5, horse 2 moves counterclockwise for 0.5 seconds, reaching position 5.0 at $t = 1.0$. The answer is 5.
- Query `1 5` (Horse 1 after 5 seconds): After several collisions, horse 1 is at position 4 at $t = 5.0$. The answer is 4.

Jealous Juniors

This year, the International Child Prodigy Coalition has admitted N new children to its special kindergarten. Over time, some of the children become friends. Friendship is mutual: if child u is friends with child v, then child v is also friends with child u. However, friendship is not necessarily transitive: if child u is friends with child v and child v is friends with child w, child u might not be friends with child w. Initially, no two children are friends.

Whenever a child performs well in the group, the teacher awards them a toy. However, the teacher does not want any child to feel discouraged, so no child can receive more than 50 toys. Initially, none of the children have any toys.

Sometimes, a child gets bored of playing with their friends and becomes jealous of other children who have many toys. In particular, the child wonders what the maximum number of toys owned by any other child who is **not** their friend is. Your task is to simulate the process of children becoming friends and receiving toys, and answer each jealous child's query.

Input

The first line of the input contains two integers N and Q ($1 \leq N, Q \leq 400\,000$) – the number of children and the number of events, respectively. The children are numbered from 1 to N.

Each of the next Q lines describes an event in one of the following formats:

- `F` i j – children i and j become friends. It is guaranteed that $i \neq j$ and that they are not already friends.
- `A` i – child i is awarded a toy. It is guaranteed that child i has strictly fewer than 50 toys before this event.
- `Q` i – child i asks for the maximum number of toys owned by any other child who is not their friend.

Output

For each query of type `Q` i, output a single integer – the maximum number of toys owned by any other child who is not a friend of child i. If child i is friends with all other children, output -1.

Examples

input	output
3 10 Q 1 A 2 A 1 A 2 Q 3 Q 2 F 2 3 Q 3 F 1 3 Q 3	0 2 1 1 -1

Explanation

In the sample:

- Initially, all children have 0 toys and no friendships exist.
- `Q 1`: The non-friends of child 1 are children 2 and 3, each having 0 toys. The maximum is 0.
- Child 2 is awarded two toys and child 1 is awarded one toy.
- `Q 3`: The non-friends of child 3 are child 1 (1 toy) and child 2 (2 toys). The maximum is 2.
- `Q 2`: The non-friends of child 2 are child 1 (1 toy) and child 3 (0 toys). The maximum is 1.
- `F 2 3`: Children 2 and 3 become friends.
- `Q 3`: The only non-friend of child 3 is child 1, who has 1 toy. The maximum is 1.
- `F 1 3`: Children 1 and 3 become friends.
- `Q 3`: Child 3 is now friends with all other children, so the answer is -1.

Kakapo Peacekeepers

Last year, there were quite a few problems with kakapos fighting each other for territory. To ~~save the kakapos~~ increase viewership and profit, a nature production company has decided to film a documentary about keeping peace among the kakapos. They have already identified N kakapo nests in the forest. The nests are located in a straight line and numbered from 1 to N from left to right. Nest i is at altitude P_i, and all nests have pairwise distinct altitudes.

The company wants to hire a team of peacekeepers to watch over the kakapos and prevent them from fighting. Each peacekeeper will be stationed at a nest and will watch over that nest, as well as every other nest visible from it. Two nests can see each other if every nest between them is at a strictly lower altitude than **both** of them. (In particular, two adjacent nests can always see each other, as there are no nests between them.) Formally, nest i and nest j $(i < j)$ can see each other if and only if

$$\max(P_{i+1}, P_{i+2}, \ldots, P_{j-1}) < \min(P_i, P_j).$$

Your task is to find the minimum number of peacekeepers that need to be hired in order to watch over all N nests.

Input

The first line contains a single integer N $(1 \leq N \leq 300\,000)$ – the number of kakapo nests.

The second line contains N integers $P_1, P_2, \ldots, P_N$ $(1 \leq P_i \leq N)$ – the altitudes of the kakapo nests. All altitudes are distinct.

Output

Output a single integer – the minimum number of peacekeepers that need to be hired in order to watch over all nests.

Examples

input	output
1 1	1
3 2 1 3	1
5 1 2 3 4 5	2

Explanation

In the second sample, the nests have altitudes $2, 1, 3$:

- Nest 1 and nest 2 are adjacent, so they can see each other.
- Nest 2 and nest 3 are adjacent, so they can see each other.
- Between nest 1 and nest 3, the only intermediate nest is nest 2 with altitude $1 < \min(2, 3)$. Thus, nest 1 and nest 3 can also see each other.

Stationing a single peacekeeper at nest 1 (or nest 3) covers all 3 nests, so the minimum number is 1.

In the third sample, the altitudes are strictly increasing $(1, 2, 3, 4, 5)$. No two non-adjacent nests can see each other. Stationing peacekeepers at nests 2 and 4 covers all 5 nests, which requires 2 peacekeepers.

Lost Signal

A long, straight hallway of length L meters extends from coordinate 0 to coordinate L.

To provide internet access in the hallway, two Wi-Fi routers have been installed:

- The first router is placed at coordinate X_1 and broadcasts with a signal radius of R_1 meters, covering the interval $[X_1 - R_1, X_1 + R_1]$.
- The second router is placed at coordinate X_2 and broadcasts with a signal radius of R_2 meters, covering the interval $[X_2 - R_2, X_2 + R_2]$.

Signals do not extend beyond the hallway's endpoints (0 and L), so any coverage outside $[0, L]$ is ignored. A point in the hallway is covered if it lies within the broadcast range of at least one of the two routers.

Your task is to compute the total length of the hallway between coordinates 0 and L that has **no** Wi-Fi coverage.

Input

The first line contains a single integer L ($1 \leq L \leq 1\,000\,000$) – the total length of the hallway in meters.

The second line contains two integers X_1 and R_1 ($0 \leq X_1 \leq L$, $0 \leq R_1 \leq L$) – the position and signal radius of the first router.

The third line contains two integers X_2 and R_2 ($0 \leq X_2 \leq L$, $0 \leq R_2 \leq L$) – the position and signal radius of the second router.

Output

Output a single integer – the total length of the hallway between 0 and L that has no Wi-Fi coverage.

Examples

input	output
10 2 2 8 1	4
10 4 3 8 4	1
10 5 0 3 1	8

Explanation

In the first sample, the hallway has length $L = 10$:

- Router 1 is at $X_1 = 2$ with radius $R_1 = 2$, covering $[0, 4]$.
- Router 2 is at $X_2 = 8$ with radius $R_2 = 1$, covering $[7, 9]$.

The uncovered portions are $[4, 7]$ (length 3) and $[9, 10]$ (length 1), giving a total uncovered length of $3 + 1 = 4$.

In the second sample, the two routers cover $[1, 7]$ and $[4, 12]$. Within the hallway $[0, 10]$, their combined coverage is $[1, 10]$ (length 9), leaving only $[0, 1]$ uncovered (length 1).

In the third sample, the first router is at $X_1 = 5$ with radius $R_1 = 0$, covering only the single point $[5, 5]$ (which has length 0). The second router covers $[2, 4]$ (length 2). Thus, $10 - 2 = 8$ meters of the hallway remain uncovered.

Masster of Puppets

The famous alchemist Flamel is working on the creation of the Philosopher's Stone. His early experiments have revealed that the stone will form as soon as he accumulates at least M units of gold at the same time through the transmutation of metal pieces.

Flamel has N pieces of metal to work with, numbered from 1 to N. The pieces are not uniform: transmuting the i-th piece takes S_i minutes. Each transmutation yields exactly 1 unit of gold upon completion. Flamel can only transmute one piece of metal at a time, but he may choose which pieces to transmute and in what order, working without pause between transmutations.

However, the transmutation process is unstable, and every piece of gold immediately starts to decay upon its creation at a constant rate of $1/D$ units per minute, where D is a positive integer. That is, a piece of gold weighs $1 - t/D$ units t minutes after its creation ($0 \leq t \leq D$). After D minutes, its weight reaches 0 and does not decrease further.

The Philosopher's Stone is created if, at any moment, the total weight of all existing gold pieces is at least M units. Your task is to find the minimum number of metal pieces that Flamel needs to transmute in order to create the Philosopher's Stone, or report that it is impossible.

Input

The first line of the input contains three integers N, D, and M ($1 \leq N, D, M \leq 100\,000$) – the number of metal pieces, the decay duration parameter D, and the required total weight of gold M.

The second line contains N integers $S_1, S_2, \ldots, S_N$ ($1 \leq S_i \leq 100\,000$) – the time in minutes required to transmute each piece of metal.

Output

Output a single integer – the minimum number of metal pieces Flamel needs to transmute to create the Philosopher's Stone, or -1 if it is impossible.

Examples

input	output
3 4 2 1 5 2	3
4 5 2 1 2 4 6	3
5 14 3 1 2 4 6 8	4
5 12 3 2 2 3 3 4	5
5 10 3 2 2 3 3 4	-1

Explanation

In the first sample, $D = 4$ and $M = 2$. Flamel can choose to transmute pieces with durations 5, 2, and 1 minutes in that order:

- At minute 5: The first piece finishes (weight 1.0).
- At minute 7 (2 minutes later): The second piece finishes (weight 1.0). The first piece has decayed to $1.0 - 2/4 = 0.5$. The total weight is 1.5.
- At minute 8 (1 minute later): The third piece finishes (weight 1.0). The second piece has decayed to $1.0 - 1/4 = 0.75$, and the first piece has decayed to $0.5 - 1/4 = 0.25$. The total weight is $1.0 + 0.75 + 0.25 = 2.0$.

At minute 8, the total weight reaches $M = 2$. It is impossible to achieve a total weight of at least 2 using fewer than 3 pieces.

In the fifth sample, even if Flamel transmutes all available pieces in optimal order, the total weight never reaches $M = 3$, so the output is -1.